



DEPARTMENT OF
SOFTWARE TECHNOLOGY

CCPROG1 T2 AY 2021-2022

Machine Project Specifications – Battle Rats

Deadline: June 20, 2022 (Monday) 8:00 AM

Percentage: 20%

OVERVIEW

Battle Rats is a two-player text-based strategy game loosely based on the mobile game, Battle Cats, made by PONOS (<https://battlecats.club/en/series/battlecats/>).

Set in the Medieval Era, the players are acting as the military advisors of two opposing rat kingdoms, *Rodentopolis* and *Squeaktopia*. The objective of the game is to stand stronger than the opponent at the end of the medieval era **OR** destroy the enemy's castle. To achieve such a laborious task, the players must **enlist** the help of a rodent army with the aim of sending an army stronger than the enemy's. However, the stronger the unit is, the more expensive it will be.

As such, to keep up with the costs of forming armies, the victorious will be awarded an amount based on the composition of the enemy's defeated army. Fortunately, the remigods (rat demigods) are charitable and may occasionally trigger a **golden age** for the struggling kingdom. The recipient of a golden age is given a boost in their economy as well as reduced costs in enlisting their units.

The following sections describe the mechanics of the game in more detail:

Player Resources

Both players are given 3000 coins (in **integers**) and a castle health of 1300 HP (hit points, in **integers** as well) at the beginning of the game. The players must manage the use of coins wisely, as they will not earn sufficiently unless they win a battle or be a recipient of a golden age.

Rodentopolis will be played by Player 1 and *Squeaktopia* will be played by Player 2.

The name of the player's kingdom, the current amount of coins, and the current health of the player's castle should be displayed during the player's turn.

Rounds and Turns

Each round in Battle Rats is quantified in decades (10 years). The game starts in the year 1300, and ends on the year 1400, with it being the final year the players will play a round on. For every round, each of the players are given their own turns. The current year should be displayed during the player's turn.

During a turn, the player is prompted with four actions. They can:

1. enlist a footmouse/mice,
2. enlist a rat knight(s),
3. enlist the rat king, or
4. rally their army, which ends their turn.

After the first player rallies their army, the second player will then be given the same prompt. After both players have rallied their army, the victorious army will then be determined based on the total strength of the armies. Once the result of the battle is decided, the round ends and the current year progresses to the next decade. Both players are awarded 500 coins at the end of every round.

Enlist

In a player's turn, the player can choose to enlist either footmice, rat knights, or the rat king. Each unit type vary in strength, cost, and also the reward that unit provides the enemy player for winning a battle. Table 1 below summarizes the different properties of each unit type.

Name	Strength	Cost	Reward
Footmouse	5	100	110
Rat Knight	35	500	650
The Rat King	300	5000	5500

Table 1. Strength, cost, and reward for each unit type.

The player can enlist multiple footmice and rat knights at a time, however they can only enlist one rat king at a time. They can enlist as much as their economy (amount of coins) allows them to. For both footmouse and rat knight options, the player will be asked for the quantity of that unit type they wish to enlist. *The program should display an error message if the player cannot afford that amount.* After displaying the message, the game should go back to the four primary actions prompt.

The current number of enlisted units per unit type in the army should be displayed in the entirety of the player's turn.

Rally (Battle)

The two players march their enlisted armies forward. The battle for a round will begin once the second player ends their turn (by selecting the rally option). To determine the outcome of the battle, both armies' **total strength** is compared, with the higher strength army becoming the victor. *The total strength should be displayed the entire time during the player's turn.*

The winner is given the opportunity to damage the enemy's castle, and the amount of damage the winner will deal is based on the **difference** between the two armies' total strengths.

In addition to this, the winner of the battle is also awarded *the spoils of the battle*. The spoils of the battle is the amount of coins the winner is awarded. The amount of coins is based on the **total reward** of the enemy army's composition.

During this phase (Battle), both players' army composition and total strength should be displayed. The winner of the battle is displayed, as well as:

1. the amount of rewards the winning player will gain, and
2. the amount of damage (strength difference) it will deal to the losing player's castle.

An example screen for the Battle Screen is described in Appendix B.

An example calculation of the total strength, total reward, and the damage dealt to the enemy's castle are described in Appendix A.

After a battle, both players' armies will all retire and will not be a part of the next decade's (round's) battle.

Golden Age

A golden age is a special turn which only occurs every 40 years (every 4 rounds). This means that for the duration of a standard game (i.e., from 1300 to 1400), a golden age occurs on the years 1340 and 1380. A golden age is only given to one player at a given instance, and is only given to the *struggling player*.

The *struggling player* is decided through a **score system** based from the amount of HP left in their castle as well as the amount of coins they currently have available. The player with a lower score is deemed as the *struggling player*. The formula for calculating the score is as follows:

$$\text{score} = \text{castleHP} + (0.33 * \text{coins})$$

Effects of a Golden Age

During a golden age, the struggling player is given 1500 to 3000 coins¹ at the beginning of the round, as well as a 20% discount (*the discounted price should be in integers*) on all of their enlists costs for that round. The discount no longer applies on the following round. The amount of coins awarded to the struggling player at the beginning of the golden age round should also be displayed during the struggling player's turn.

Game Over

The game can end through either of two scenarios:

1. when a players castle loses all of their hp or the end of the medieval era is reached.
2. when the game reaches the end of the era (i.e., after the round of year 1400), the winner is decided based on the same **score system** described earlier. The player with the higher score wins.

The final scores of both players, and the winner of the game should be displayed.

DELIVERABLES

¹ This requires the use of random. For information on how to use the random functions in C, refer to the topic page on pre-defined functions, under the Standard Library section (stdlib.h)

1. Source Code (.c file(s))

This is the source code for the Machine Project. The final project must be written in the C programming language. Source code written in another programming language (e.g., C++), **will not be accepted**.

The source code should make use of iterative statements, conditionals, and user-defined functions. The source code may be consisted of multiple .c and .h files. The .c file that contains the main program should include the statement of Academic Honesty at the beginning of the file, written as a comment:

```
/*
*****
This is to certify that this project is my own work, based on my personal
efforts in studying and applying the concepts learned. I have constructed
the functions and their respective algorithms and corresponding code by
myself. The program was run, tested, and debugged by my own efforts. I
further certify that I have not copied in part or whole or otherwise
plagiarized the work of other students and/or persons.
<Your Full Name> - <ID Number> - <Section>
*****
*/
```

2. Test Cases Document (.pdf)

For each function, identify appropriate test cases/scenarios. Test and validate your functions. Use the following table format for every function.

Function: *function declaration/signature*

Test Description	Input value/parameters	Expected output/result	Actual output/result	Pass/Fail
<i>Case 1 desc</i>				P or F
<i>Case 2 desc</i>				
:				

Example:

Function: ***addTwoNumbers (int numA, int numB)***

Test Description	Input value/parameters	Expected output/result	Actual output/result	Pass/Fail
<i>Input positive numbers</i>	numA = 15, numB = 135	148	148	P
<i>Input negative numbers</i>	numA = -153, numB = -32	-185	-185	P
:				

Other Requirements

Documentation and Coding Standards:

1. Follow coding standards stated in the [Coding Standards](#) page from Canvas.
2. Include internal documentations, especially on snippets of code that are not easily understandable/a bit complex.
3. Include function specifications in the source code. Before each of your functions, include the comments containing function specifications. Follow the format below.

```
/*
    Description:  what the function does
    Parameters:
        param1    what this param1 is for
        param2    description of param2
        :
    Return value: description
*/
```

Screens:

1. It is up to the programmer how each screen will look like, as long as the required information are displayed.
 - a. The required information are colored in the specifications above as **purple**.
 - b. The screens must be neat and organized
2. This is a menu-based application. The inputs from the user should ideally be whole numbers and/or characters. Refer to Appendix B for example screens.

Error Handling:

The program should handle invalid inputs—it should display an appropriate error message and ask the user for another input. It should keep on asking until a valid input has been received.

Examples of invalid inputs are (but not limited to):

- Input for Menu Navigation, wherein the input was not part of the options.
- Trying to enlist more units than what the player can afford.
- Entering a negative value for the quantity of units to enlist.

IMPORTANT NOTES

1. All source codes (with internal documentation and function specs) and Test Case documents (pdf file) MUST be uploaded in AnimoSpace **on or before** the due date.
2. You may submit multiple times, but only the last submission will be considered.
3. For all submissions made, the following are assumed:
 - a. You worked on the entire project by yourself;
 - b. the submitted files contain your original work; and
 - c. you did not share a part or the entire source code to other people.
4. This is an **individual** project. Any form of cheating (working in collaboration, asking other person's help to accomplish a part or the entire MP, copying any code from another person's work or post) will merit a grade of 0.0 for the CCPROG1 and a discipline case.
5. You will present your project during a scheduled demo. The schedule will be announced later. During the demo, the submitted code will be **downloaded, compiled, and executed**.
6. Your solution **must** include use of functions, conditional, and iterative statements.
7. You will **incur deductions** when:
 - a. The return statement used in a void function.
 - b. Minimal use of user-defined functions.
 - c. Features are either missing or not fully implemented, or instructions are not followed.
 - d. Multiple warnings appear upon compiling. To enable warnings in your compiler, use gcc -Wall or Dev-C++'s -Wall compiler option.
8. The following will **automatically make your MP score 0**:
 - a. Not submitting before the deadline. AnimoSpace flagged your submission as **LATE**.
 - b. Use of global variables.
 - c. Use of keywords such as goto, continue, and statements such as System.exit().
 - d. Calling the main() function to make the program execute again.
 - e. Use of break statements in a non-switch block.
 - f. No user-defined functions are used. Your program is just one big main() function.
 - g. Compilation error encountered during the demo.
 - h. The program is not compiled in C (.cpp programs are not considered as valid)
 - i. No visible outputs on the screen.

APPENDIX A: EXAMPLE CALCULATIONS

Total Strength Calculation

For reference, the strength values are:

Footmouse: 5, Rat Knight: 35, Rat King: 300

Player 1 Army	Player 2 Army
# of footmice: 3 ($3 \times 5 = 15$)	# of footmice: 5 ($5 \times 5 = 25$)
# of rat knights: 4 ($4 \times 35 = 140$)	# of rat knights: 0 ($0 \times 35 = 0$)
rat king: 0 ($0 \times 300 = 0$)	rat king: 1 ($1 \times 300 = 300$)
Total strength: 155	Total strength: 325

In this scenario, Player 2 will win the battle as they have higher total strength compared to Player 1. The amount of damage player 2 will deal to player 1's castle is: $325 - 155 = 170$.

So if player 1 currently has 1300 castle HP, after the battle, their castle will drop down to 1130 ($1300 - 170$).

Rewards Calculation

For reference, the reward values are:

Footmouse: 110, Rat Knight: 650, Rat King: 5500

Given that player 2 wins the battle, we are going to refer to player 1's army composition for the total reward given to player 2.

Lost Player's Army (Player 1)	Reward equivalent
# of footmice: 3	$3 \times 110 = 330$
# of rat knights: 4	$650 \times 4 = 2600$
rat king: 0	$0 \times 5500 = 0$
	Total Reward: 2930

Player 2 will be awarded 2930 coins after the battle.

APPENDIX B: SAMPLE SCREENS

Note: you are not limited to this layout, you may decide on your own layout as long as the **required information** is displayed.

Text highlighted in **yellow** denotes user input. Text highlighted in **blue** are comments/additional information.

Player Actions

```
Year 1300
Rodentopolis' (Player 1's) Turn
-----
Castle HP: 1300
Coins: 3000
Army:
# of Footmice: 0
# of Rat Knights: 0
Rat King: 0
-----
What would you like to do?
1 - Enlist Footmice (100 coins each)
2 - Enlist Rat Knight (500 coins each)
3 - Enlist Rat King (5000 coins)
4 - Rally Troops
-----
Enter a number (1-4):
1
How many footmice would you like to enlist?
3
You enlisted 3 footmice for 300 coins.
=====
Castle HP: 1300
Coins: 2700
Army:
# of Footmice: 3
# of Rat Knights: 0
Rat King: 0
-----
What would you like to do?
1 - Enlist Footmice (100 coins each)
2 - Enlist Rat Knight (500 coins each)
3 - Enlist Rat King (5000 coins)
4 - Rally Troops
-----
Enter a number (1-4):
5
Invalid input!
-----
What would you like to do?
1 - Enlist Footmice (100 coins each)
```

- 2 - Enlist Rat Knight (500 coins each)
- 3 - Enlist Rat King (5000 coins)
- 4 - Rally Troops

Enter a number (1-4):

2

How many Rat Knights would you like to enlist?

4

You enlisted 4 Rat Knights for 2000 coins.

=====

Castle HP: 1300

Coins: 700

Army:

of Footmice: 3

of Rat Knights: 4

Rat King: 0

What would you like to do?

- 1 - Enlist Footmice (100 coins each)
- 2 - Enlist Rat Knight (500 coins each)
- 3 - Enlist Rat King (5000 coins)
- 4 - Rally Troops

Enter a number (1-4):

4

You have rallied your troops:

Army:

of Footmice: 3

of Rat Knights: 4

Rat King: 0

Player 1 should look away, it's Player 2's turn!

=====

.
.
(tons of newlines here so that player 2 won't see what player 1 did)

.

Year 1300

Squeaktopia's (Player 2's) Turn

Castle HP: 1300

Coins: 3000

Army:

of Footmice: 0

of Rat Knights: 0

Rat King: 0

What would you like to do?

- 1 - Enlist Footmice (100 coins each)
- 2 - Enlist Rat Knight (500 coins each)

```
3 - Enlist Rat King (5000 coins)
4 - Rally Troops
-----
Enter a number (1-4):
... (same process as player 1)
```

Battle Screen

```
It is the year 1300, and both kingdoms have rallied their troops!
=====
Rodentopolis
Castle HP: 1300
Army:
# of Footmice: 3
# of Rat Knights: 4
Rat King: 0
-----
Squeaktopia
Castle HP: 1300
Army:
# of Footmice: 5
# of Rat Knights: 2
Rat King: 0
=====
Rodentopolis Total Strength: 155
Squeaktopia Total Strength: 95
-----
Rodentopolis Wins!
Rodentopolis deals 60 damage to Squeaktopia's castle (new HP: 1240)
=====
As the winner, Rodentopolis gains a total of 1850 coins (new Coins: 2550) .
End of battle.
=====
Post Battle Summary:
-----
Rodentopolis
Castle HP: 1300
Coins: 2550
-----
Squeaktopia
Castle HP: 1300
Coins: 1500 ( 1500 because squeaktopia spent a total of 1500 coins during
their turn: (5 * 100 + 2 * 500) )
=====
=====
Year 1310
Rodentopolis' (Player 1's) Turn
-----
```

Castle HP: 1300

Coins: 2550

Army:

of Footmice: 0

of Rat Knights: 0

Rat King: 0

.

.

. (continue same process as Player Actions)